

Payroll Review API — Инструкция по тестированию

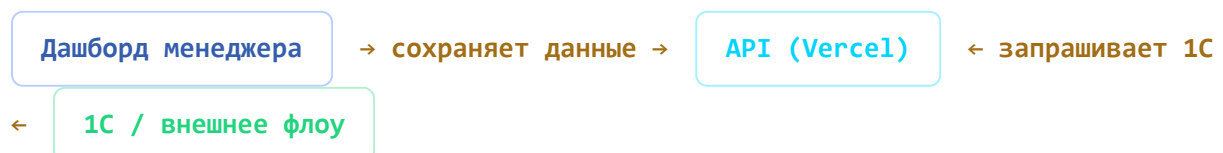
Версия ПР-9.3.2 · 30.06.2026 · [Swagger UI](#) · [OpenAPI Spec](#)

Содержание

1. Архитектура: как всё связано
2. Что нужно для тестирования
3. Аутентификация
4. Шаг 0: Проверка доступности
5. Шаг 1: Справочник проектов
6. Шаг 2: Справочник разработчиков
7. Шаг 3: Зарплатный снапшот
8. Шаг 4: Выгрузка для 1С (billing)
9. Шаг 5: Отметка обработанных задач
10. Шаг 6: Проверка защиты от дублей
11. Полный сценарий «как будет работать 1С»
12. Справочник ошибок
13. FAQ

1. Архитектура: как всё связано

Твоя метафора точная. Вот как это работает:



Дверь 1 — Дашборд менеджера (obmen-atilab.vercel.app)

- Менеджер видит часы, ставки, задачи
- Нажимает «Сохранить» → данные летят в API → сохраняются в Vercel Blob
- **Без сохранённого снапшота API вернёт 404** — это частая ошибка при тестировании

Мостик — API (эндпоинты на Vercel)

- Хранит снапшоты и processed-файлы в Vercel Blob Storage
- Отдаёт данные по запросу (GET)
- Принимает обратную связь от 1С (POST mark-processed)

Дверь 2 — 1С / внешнее флоу

- Запрашивает: какие задачи в «Счёт» и «Оплата»?
- Формирует счета клиентам и выплаты разрабам
- Сообщает API: «я обработал задачи 123, 456» — чтобы не выгрузить повторно

ⓘ API не меняет стадии задач в Bitrix24. Перевод задач по стадиям делает менеджер вручную. API только читает текущее состояние.

2. Что нужно для тестирования

Обязательное

- ☐ Доступ в интернет
- ☐ API-ключ: pr_api_2026
- ☐ Любой HTTP-клиент (curl, Postman, браузер, 1С HTTPЗапрос)

Рекомендуемое

- ☐ [Postman](#) — удобнее чем curl, есть GUI
- ☐ Браузер — для Swagger UI: obmen-atilab.vercel.app/docs/
- ☐ Доступ к дашборду — чтобы сохранить снапшот перед тестом

Критически важное условие

⚠ Перед тестированием API менеджер ДОЛЖЕН открыть дашборд и нажать «Сохранить». Без сохранённого снапшота эндпоинты /api/payroll, /api/developers, /api/payroll/.../billing вернут **404 No data found**. Если видишь 404 — это не баг, это

значит что данных нет. Зайди в дашборд → дождись загрузки → нажми «Сохранить» → повтори запрос.

3. Аутентификация

Все эндпоинты (кроме прокси Bitrix24) требуют API-ключ. Два способа передачи:

Способ 1: Query-параметр

`GET https://obmen-atilab.vercel.app/api/projects?key=pr_api_2026`

Способ 2: HTTP-заголовок

`GET https://obmen-atilab.vercel.app/api/projects`
`X-API-Key: pr_api_2026`

❗ Для 1С рекомендуем заголовок X-API-Key — ключ не логируется в URL и не попадает в access-логи.

4. Шаг 0: Проверка доступности

Перед тестированием API убедись что сервер отвечает.

0 Проверить что сервер жив

```
curl -s -o /dev/null -w "%{http_code}" https://obmen-atilab.vercel.app/api/projects?
```



Ожидаемый результат:

200

Если 200 — сервер работает, API доступен, ключ верный. Идём дальше.

Если не 200:

403 → неверный API-ключ (проверь `key=pr_api_2026`)
000 / `ERR_CONNECTION` → сервер недоступен (проверь VPN/доступ)
502 / 503 → Vercel перезапускается, подожди 30с и повтори

5. Шаг 1: Справочник проектов

GET Самый простой эндпоинт. Не зависит от снапшота — данные статичны. Идеален для первой проверки.

1 Получить список проектов со стадиями

```
curl -s "https://obmen-atilab.vercel.app/api/projects?key=pr_api_2026" | python3 -m
```

**Ожидаемый результат:**

```
{ "generatedAt": "2026-06-30T10:00:00.000Z", "projectsCount": 22, "projects": [ {  
  "projectId": 4, "projectName": "Живое пиво", "stagesCount": 10, "stages": [  
    { "id": 80, "name": "Новые", "paymentStatus": "not_ready" }, ... { "id": 408,  
    "name": "Счет", "paymentStatus": "invoice" }, { "id": 410, "name": "Оплата",  
    "paymentStatus": "paid" } ], "invoiceStageId": 408, "paidStageId": 410,  
    "closedStageId": 84 }, ... ], "paymentStatusLegend": { ... } }
```

Что проверяем:

- ☐ `projectsCount > 0` — проекты есть
- ☐ У каждого проекта есть `invoiceStageId` и `paidStageId` — это ID стадий «Счёт» и «Оплата»
- ☐ `paymentStatus` принимает только значения: `not_ready`, `invoice`, `paid`, `closed`
- ☐ Стадия «Счёт» = `invoice`, «Оплата» = `paid` — именно они нужны 1С

6. Шаг 2: Справочник разработчиков

GET Реквизиты разработчиков для 1С (ИНН, банк, договор). Зависит от сохранённого снапшота!

2 Получить список разработчиков

```
curl -s "https://obmen-atilab.vercel.app/api/developers?key=pr_api_2026" | python3 -i
```



Ожидаемый результат (если снапшот сохранён):

```
{ "period": "2026-06", "savedAt": "2026-06-25T10:30:00.000Z", "developers": [ {  
  "devId": 82, "fullName": "Иванов Иван Иванович", "inn": "7700000000",  
  "selfEmployed": "Самозанятый", "bank": "Тинькофф", "contract": "Договор №123",  
  "contractDate": "2026-01-15", "rate": 1000, "clientRate": 1800, "base": 80000,  
  "active": true }, ... ] }
```

Если результат 404:

```
{ "error": "No data found", "hint": "Manager needs to save data first from the  
payroll dashboard" }
```

→ **Это нормально если снапшот не сохранён.** Зайди в дашборд, дождись загрузки данных, нажми «Сохранить», повтори запрос.

Что проверяем:

- ☐ developers — непустой массив
- ☐ У каждого разработчика есть inn, bank, contract — реквизиты для 1С
- ☐ rate — ставка разработчика (руб./час), clientRate — для клиента
- ☐ active: true — разработчик активен в текущем месяце

7. Шаг 3: Зарплатный снапшот

GET Полные данные за месяц. Два режима: summary (сводка) и details (детализация).

За Получить сводку за месяц

```
curl -s "https://obmen-atilab.vercel.app/api/payroll/2026-06?key=pr_api_2026" | python
```



Ожидаемый результат:

```
{ "period": "2026-06", "savedAt": "2026-06-25T10:30:00.000Z", "version": 1,
  "developers": [ ... ], "totals": { "totalHours": 480.5, "totalPayroll": 480500,
  "totalClientAmount": 864900 } }
```

36 Получить детализацию

```
curl -s "https://obmen-atilab.vercel.app/api/payroll/2026-06?key=pr_api_2026&view=de
```



Отличие от summary: вместо totals будет массив details — по каждой задаче каждого разработчика:

```
{ "period": "2026-06", "developers": [ ... ], "details": [ { "taskId": 7114,
  "taskTitle": "Разработка модуля отчётности", "devId": 82, "projectId": 4,
  "billableHours": 24.5, "payrollAmount": 24500, "clientAmount": 44100, "stageId":
  408, "stageName": "Счет", "paymentStatus": "invoice" }, ... ] }
```

Что проверяем:

- ☐ details[] содержит задачи с paymentStatus
- ☐ Задачи в «Счёт» имеют paymentStatus: "invoice"
- ☐ Задачи в «Оплата» имеют paymentStatus: "paid"
- ☐ Задачи в других стадиях имеют paymentStatus: "not_ready" или "closed"

8. Шаг 4: Выгрузка для 1С (billing) — главный эндпоинт

GET Это то, ради чего всё построено. Возвращает задачи, готовые к финансовой обработке.

4 Получить счета и выплаты

```
curl -s "https://obmen-atilab.vercel.app/api/payroll/2026-06/billing?key=pr_api_2026"
```

Ожидаемый результат:

```
{ "period": "2026-06", "generatedAt": "2026-06-30T10:00:00.000Z", "invoices": [
... ], ← задачи на стадии «Счёт» – 1С формирует счёт клиенту "payouts": [ ... ],
← задачи на стадии «Оплата» – 1С формирует акт + выплату разрабу "totals": {
"invoicesCount": 5, "invoicesAmount": 125000, ← сумма к выплате разрабам (счета)
"invoicesClientAmount": 225000, ← сумма для клиентов (счета) "payoutsCount": 3,
"payoutsAmount": 78000, ← сумма к выплате разрабам (оплаты)
"payoutsClientAmount": 140400, ← сумма для клиентов (оплаты) "processedCount": 0
← уже обработано (двойная защита) } }
```

Структура каждого элемента invoices/payouts:

Поле	Тип	Описание
<code>taskId</code>	int	ID задачи в Bitrix24
<code>taskTitle</code>	string	Название задачи
<code>projectId</code> / <code>projectName</code>	int / string	Проект
<code>stageId</code> / <code>stageName</code>	int / string	Стадия
<code>developer</code>	object	ФИО, ИНН, банк, договор
<code>hours</code>	float	Биллинговые часы
<code>rate</code>	int	Ставка разраба (руб./час)
<code>amount</code>	int	Сумма к выплате разрабу
<code>clientRate</code>	int	Ставка для клиента (руб./час)
<code>clientAmount</code>	int	Сумма для выставления клиенту

Что проверяем:

- ☐ `invoices` — задачи именно со стадии «Счёт» (`paymentStatus = invoice`)
- ☐ `payouts` — задачи именно со стадии «Оплата» (`paymentStatus = paid`)
- ☐ Каждый элемент содержит `developer` с реквизитами

- ☐ `amount = hours × rate` (сумма к выплате разработчику)
- ☐ `clientAmount = hours × clientRate` (сумма для клиента)
- ☐ Задачи, которые уже были отмечены через `mark-processed`, **не появляются**

⚠ Если `invoices` и `payouts` пустые — это может быть нормально: нет задач в стадиях «Счёт» и «Оплата». Менеджер должен перевести хотя бы одну задачу в «Счёт» в Bitrix24, потом пересохранить снимок.

9. Шаг 5: Отметка обработанных задач

POST Обратная связь от 1С: «я обработал эти задачи, не выгружай повторно».

5 Отправить список обработанных задач

```
curl -X POST "https://obmen-atilab.vercel.app/api/admin/mark-processed?key=pr_api_20" \
-H "Content-Type: application/json" \
-d '{
  "period": "2026-06",
  "items": [
    {"taskId": 7114, "action": "invoice_created", "processedAt": "2026-06-30T10:00"},
    {"taskId": 7916, "action": "paid_out", "processedAt": "2026-06-30T10:05:00Z"}
  ]
}'
```

Ожидаемый результат:

```
{ "ok": true, "period": "2026-06", "itemsCount": 2, "url": "https://...payroll-processed-2026-06.json", "updatedAt": "2026-06-30T10:10:00.000Z" }
```

Параметры action:

action	Когда вызывать	Описание
<code>invoice_created</code>	1С сформировала счёт клиенту	Для задач из <code>invoices[]</code>
<code>paid_out</code>	1С провела акт + выплату разрабу	Для задач из <code>payouts[]</code>

Что проверяем:

- ☐ Ответ ok: true
- ☐ itemsCount совпадает с количеством отправленных задач
- ☐ Повторный вызов с теми же taskId обновляет записи (не дублирует)

10. Шаг 6: Проверка защиты от дублей

Самый важный тест для 1С — убедиться что обработанные задачи не появятся снова.

6a Вызвать billing повторно

```
curl -s "https://obmen-atilab.vercel.app/api/payroll/2026-06/billing?key=pr_api_2026"
```

Ожидаемый результат:

- Задачи с taskId: 7114 и taskId: 7916 **не должны появиться** в invoices/payouts
- totals.processedCount должен быть ≥ 2

6б Проверить что НОВЫЕ задачи всё ещё видны

Если менеджер перевёл ещё одну задачу в «Счёт» и пересохранил снимок — она должна появиться в invoices.

Защита от дублей работает **только для обработанных задач**. Новые задачи видны всегда.

11. Полный сценарий «как будет работать 1С»

Ежемесячный цикл: 1. Менеджер работает в дашборде весь месяц - Видит часы, задачи, ставки - Переводит задачи по стадиям в Bitrix24 - Нажимает «Сохранить» → снимок летит в API 2. В конце месяца 1С запускает выгрузку: →

```
GET /api/payroll/2026-06/billing
```

← Получает invoices[] (счета клиентам) +

payments[] (выплаты разрабам) 3. 1С обрабатывает каждую задачу: invoices[] → формирует счёт клиенту payments[] → формирует акт + выплату разрабу 4. 1С подтверждает обработку: → **POST /api/admin/mark-processed** ← Отправляет taskId + action для каждой обработанной задачи 5. При следующей выгрузке обработанные задачи не появятся (защита от двойного выставления счетов)

! API НЕ триггерит смену стадий в Bitrix24. Менеджер переводит задачи вручную. API только читает текущее состояние. Если менеджер не перевёл задачу в «Счёт» — она не появится в invoices[], даже если работа выполнена.

12. Справочник ошибок

HTTP	error	Причина	Что делать
400	Invalid period format	Неверный формат периода	Используй YYYY-MM, например 2026-06
400	Missing period or items[]	Не указан period или items в теле POST	Проверь тело запроса: {"period":"2026-06","items": [...]}
403	Invalid API key	Неверный или отсутствующий API-ключ	Добавь ?key=pr_api_2026 или заголовок X-API-Key: pr_api_2026
404	No data found for this period	Снапшот за этот месяц не сохранён	Зайди в дашборд → дождись загрузки → нажми «Сохранить»
405	Method not allowed	Неправильный HTTP-метод	GET для чтения, POST для записи. Проверь метод запроса
500	Vercel Blob not configured	Не настроен Vercel Blob Storage	Администратор: включи Blob в Vercel Dashboard → Storage
502	Proxy error	Прокси к Bitrix24 не может достучаться	Проверь что Bitrix24 доступен. Этот эндпоинт нужен только дашборду, не 1С

502 Failed to fetch
snapshot from
storage

Снапшот есть в списке,
но файл не читается

Пересохрани снапшот из дашборда

Как интерпретировать ответы

2xx Всё хорошо

200 — запрос выполнен, данные в теле ответа.

Сверь структуру с примерами выше. Если поля `invoices/payouts` пустые — это не ошибка, просто нет задач в нужных стадиях.

4xx Ошибка на вашей стороне

Неверный запрос: неправильный ключ, формат, метод. Исправь и повтори.

Особенно часто: 404 No data found — забыл сохранить снапшот в дашборде.

5xx Ошибка на стороне сервера

Что-то сломалось на нашей стороне. Если ошибка повторяется — сообщи разработчику.

500 Vercel Blob not configured — нужна настройка Blob Storage (разовая операция).

13. FAQ

Зачем нужен mark-processed? Почему бы просто не смотреть на стадии?

Стадия в Bitrix24 показывает текущее состояние задачи. Но между «1С получил выгрузку» и «1С обработал» может пройти время. Если 1С запросит billing повторно (например, перезапуск обработки), задачи появятся снова. mark-processed — это дополнительный уровень защиты от дублей поверх стадий.

Что если менеджер перевёл задачу обратно из «Счёт» в «Работа»?

При следующем сохранении снапшота задача пропадёт из `invoices[ ]`. API всегда отражает текущее состояние снапшота. Если задача была в mark-processed но потом вернулась в «Счёт» — она появится снова (это правильно, задача требует повторной обработки).

Можно ли вызывать billing несколько раз за месяц?

Да, это штатный сценарий. 1С может запрашивать billing каждый день или раз в неделю. Каждый раз получит только необработанные задачи + новые.

Что если 1С не вызовет mark-processed?

Ничего не сломается. Просто при следующем запросе billing те же задачи появятся снова. 1С должен самостоятельно отслеживать, что уже обработал (по taskId). mark-processed — удобная, но необязательная функция.

Сколько живёт снапшот?

Снапшот хранится в Vercel Blob Storage бессрочно. Каждый новый «Сохранить» перезаписывает снапшот за тот же месяц. Данные за прошлые месяцы не удаляются автоматически.

Нужно ли 1С знать про /api/bx/*?

Нет. Это CORS-прокси для браузера. 1С может ходить в Bitrix24 напрямую (у него нет CORS-ограничений). Для 1С нужны только эндпоинты /api/payroll, /api/developers, /api/projects и /api/admin/mark-processed.

Как тестировать в Postman?

Импортируй коллекцию из Swagger: obmen-atilab.vercel.app/docs/ → кнопка «Try it out». Либо создавай запросы вручную, указывая URL + API-ключ.

Как тестировать из 1С?

Используй HTTPСоединение и HTTPЗапрос:

```
// 1С: пример GET-запроса
```

```
Соединение = Новый HTTPСоединение("obmen-atilab.vercel.app", 443, , , , Новый ЗащищенноеС  
Запрос = Новый HTTPЗапрос("/api/payroll/2026-06/billing?key=pr_api_2026");  
Ответ = Соединение.Получить(Запрос);  
ТелоОтвета = Ответ.ПолучитьТелоКакСтроку();
```

```
// 1С: пример POST-запроса (mark-processed)
```

```
Запрос2 = Новый HTTPЗапрос("/api/admin/mark-processed?key=pr_api_2026");  
Запрос2.Заголовки.Вставить("Content-Type", "application/json");  
Запрос2.УстановитьТелоИзСтроки('{ "period": "2026-06", "items": [...] }');  
Ответ2 = Соединение.ОтправитьДляОбработки(Запрос2);
```

Payroll Review API v9.3.2 · [Swagger UI](#) · [OpenAPI](#)